



ARQUITETURA DE SOFTWARE

ARQUITETURA

- Projeto em mais alto nível
- Foco **não** são mais unidades pequenas (ex.: classes)
- Mas sim unidades maiores e mais relevantes
 - ← Pacotes, módulos, subsistemas, camadas, serviços, ...
- Decisões de Projeto mais importantes em um sistema

UNIDADES DE MAIOR RELEVÂNCIA?

- Definição de relevância depende do sistema
- Exemplo: banco de dados
 - ↩ Sistema de Informações: certamente é relevante
 - ↩ Sistema de Imagens Médicas: pode não ser relevante

PADRÕES ARQUITETURAIS

- "Modelos" pré-definidos para arquiteturas de software
- Vamos estudar:
 - ← Camadas (duas e três camadas)
 - ← Model-View-Controller (MVC)
 - ← Microserviços
 - ← Orientada a Mensagens
 - ← Publish/Subscribe



SOBRE A IMPORTÂNCIA DE ARQUITETURA DE SOFTWARE

DEBATE LINUS-TANENBAUM (1992)



Criador do sistema
operacional Linux



Autor de livros e do
sistema operacional
Minix

INÍCIO DO DEBATE: MENSAGEM DO TANENBAUM (1992)

```
From: ast@cs.vu.nl (Andy Tanenbaum)
Newsgroups: comp.os.minix
Subject: LINUX is obsolete
Date: 29 Jan 92 12:12:50 GMT
```

I was in the U.S. for a couple of weeks, so I
LINUX (not that I would have said much had I
it is worth, I have a couple of comments now.

ARGUMENTO DO TANENBAUM

- Linux possui uma **arquitetura monolítica**
- Quando o melhor seria uma **arquitetura microkernel**
- Monolítico: sistema operacional é um único arquivo
 - ← Gerência de processos, memória, arquivos, etc
- Microkernel: kernel só possui serviços essenciais
 - ← Demais serviços rodam como processos independentes

RESPOSTA DO LINUS

```
From: torvalds@klaava.Helsinki.FI (Linus Benedict Torvalds)  
Subject: Re: LINUX is obsolete  
Date: 29 Jan 92 23:14:26 GMT  
Organization: University of Helsinki
```

```
Well, with a subject like this, I'm afraid I'll have to reply.
```

ARGUMENTO DO LINUS

- Em teoria, arquitetura microkernel é mais interessante
- Mas existem outros critérios que devem ser considerados
- Um deles é que o Linux já era uma realidade e não apenas uma promessa

NOVA MENSAGEM DO TANENBAUM

- "Eu continuo com minha opinião. Projetar um kernel monolítico em 1991 é um erro fundamental."
- "Agradeça por não ser meu aluno. Se fosse, você não iria tirar uma nota alta com esse design."

COMENTÁRIO DO KEN THOMPSON (UNIX)

- "Na minha opinião, é mais fácil implementar um sistema operacional com um kernel monolítico."
- "Mas é também mais fácil que ele se transforme em uma bagunça à medida que o kernel é modificado."



KEN THOMPSON PREVIU O FUTURO:

17 ANOS DEPOIS (2009) VEJA A DECLARAÇÃO DE
TORVALDS EM UMA CONFERÊNCIA DE LINUX

- "Não somos mais o kernel simples, pequeno e hiper-eficiente que imaginei há 15 anos."
- "Em vez disso, o kernel está grande e inchado. Quando adicionamos novas funcionalidades, o cenário piora."



MORAL DA HISTÓRIA: OS "CUSTOS" DE UMA DECISÃO ARQUITETURAL
PODEM LEVAR ANOS PARA APARECER ...



ARQUITETURA EM CAMADAS

ARQUITETURA EM CAMADAS

- Sistema é organizado em camadas, de forma hierárquica
- Camada n somente pode usar serviços da camada $n-1$
- Muito usada em redes computadores e sistemas distribuídos

OSI model		
Layer	Name	Example protocols
7	Application Layer	HTTP, FTP, DNS, SNMP, Telnet
6	Presentation Layer	SSL, TLS
5	Session Layer	NetBIOS, PPTP
4	Transport Layer	TCP, UDP
3	Network Layer	IP, ARP, ICMP, IPSec
2	Data Link Layer	PPP, ATM, Ethernet
1	Physical Layer	Ethernet, USB, Bluetooth, IEEE802.11

VANTAGENS

1. Facilita o entendimento, pois quebra a complexidade do sistema em uma estrutura hierárquica
2. Facilita a troca de uma camada por outra (ex: TCP, UDP)
3. Facilita o reuso de uma camada (ex: várias aplicações usam TCP).

VARIAÇÕES PARA SISTEMAS DE INFORMAÇÕES

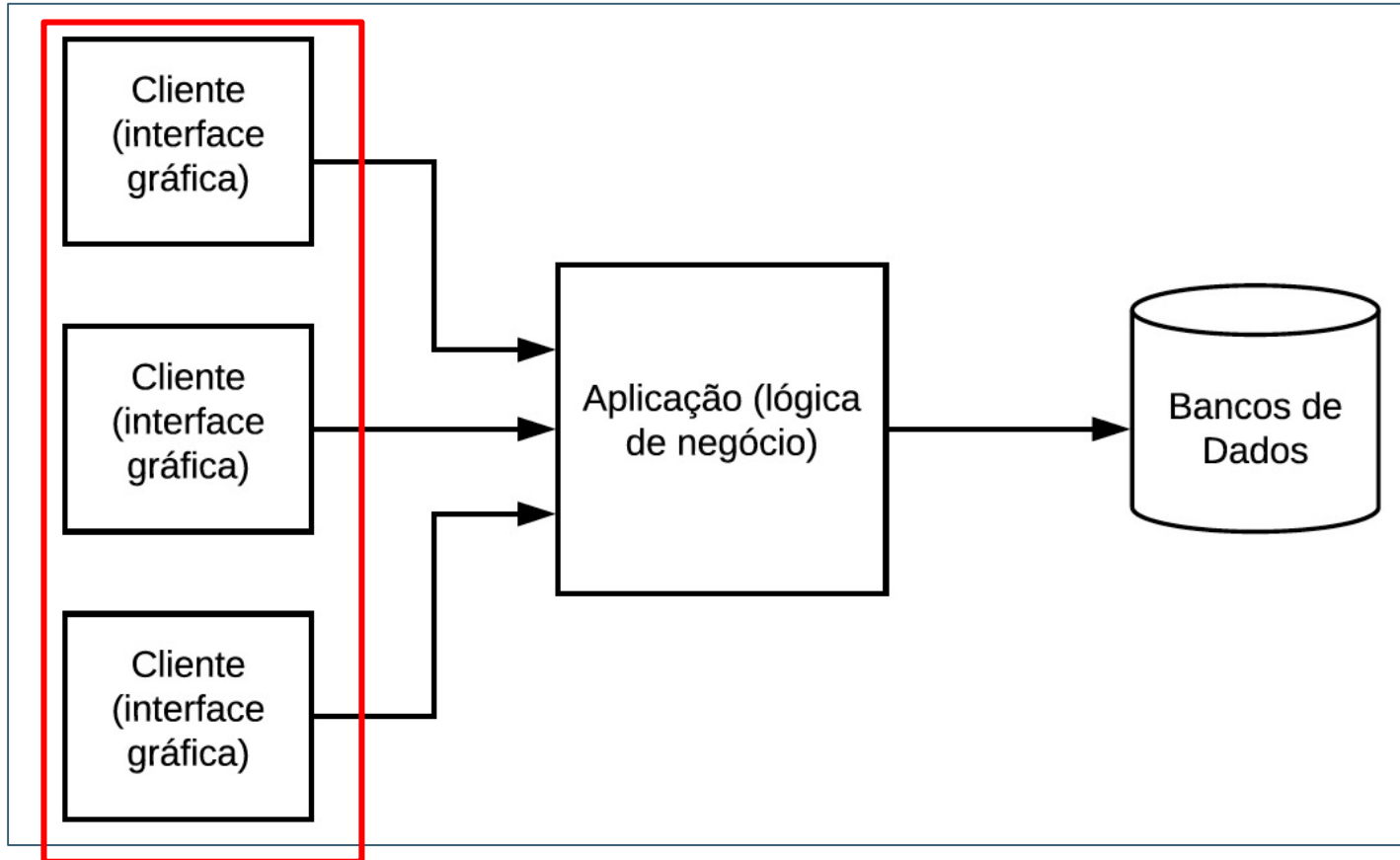
- Três camadas
- Duas camadas

ARQUITETURA EM TRÊS CAMADAS

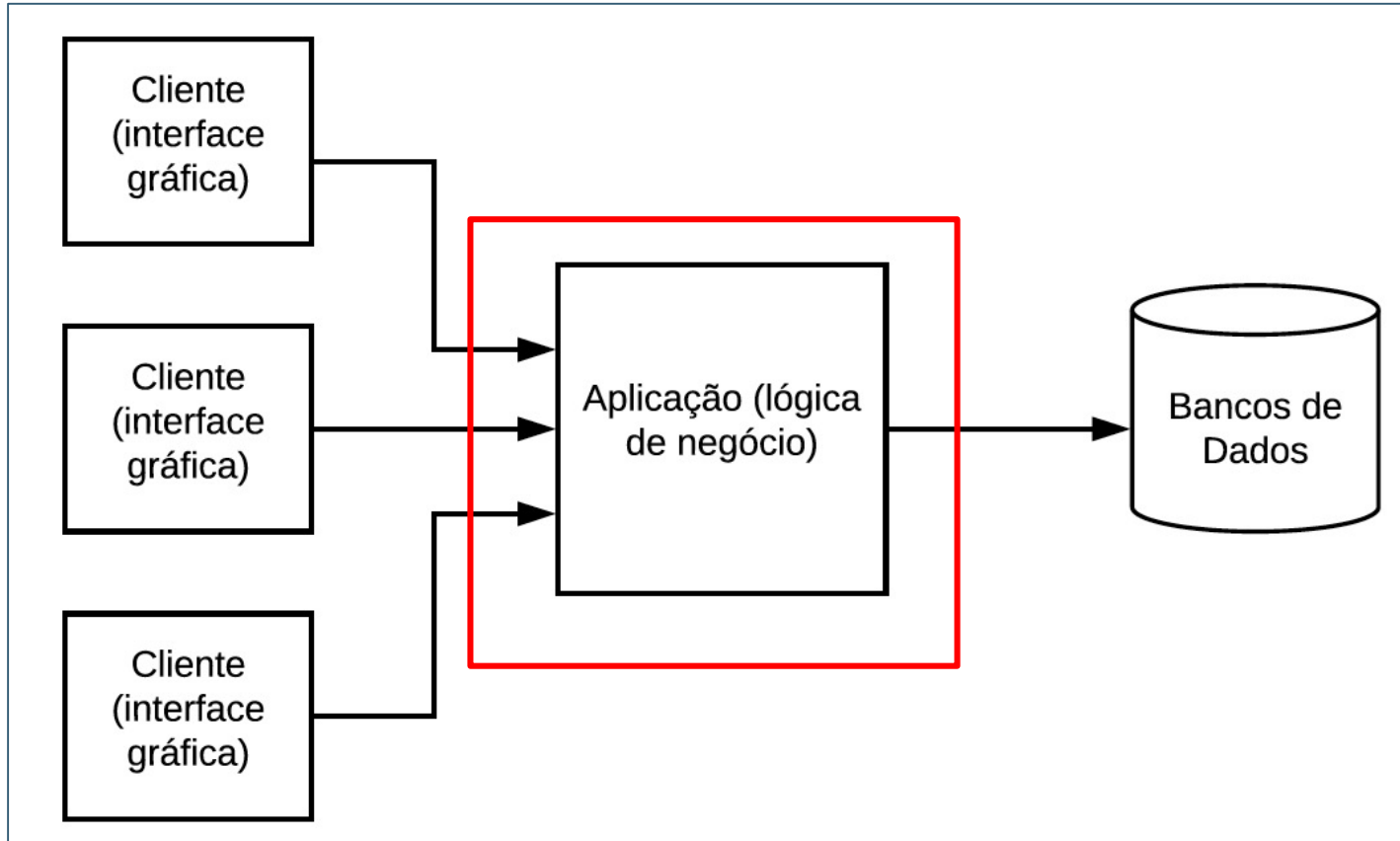
- Comum em processos de "downsizing" de aplicações corporativas nas décadas de 80 e 90
- Downsizing: migração de computadores de mainframes para servidores, rodando Unix



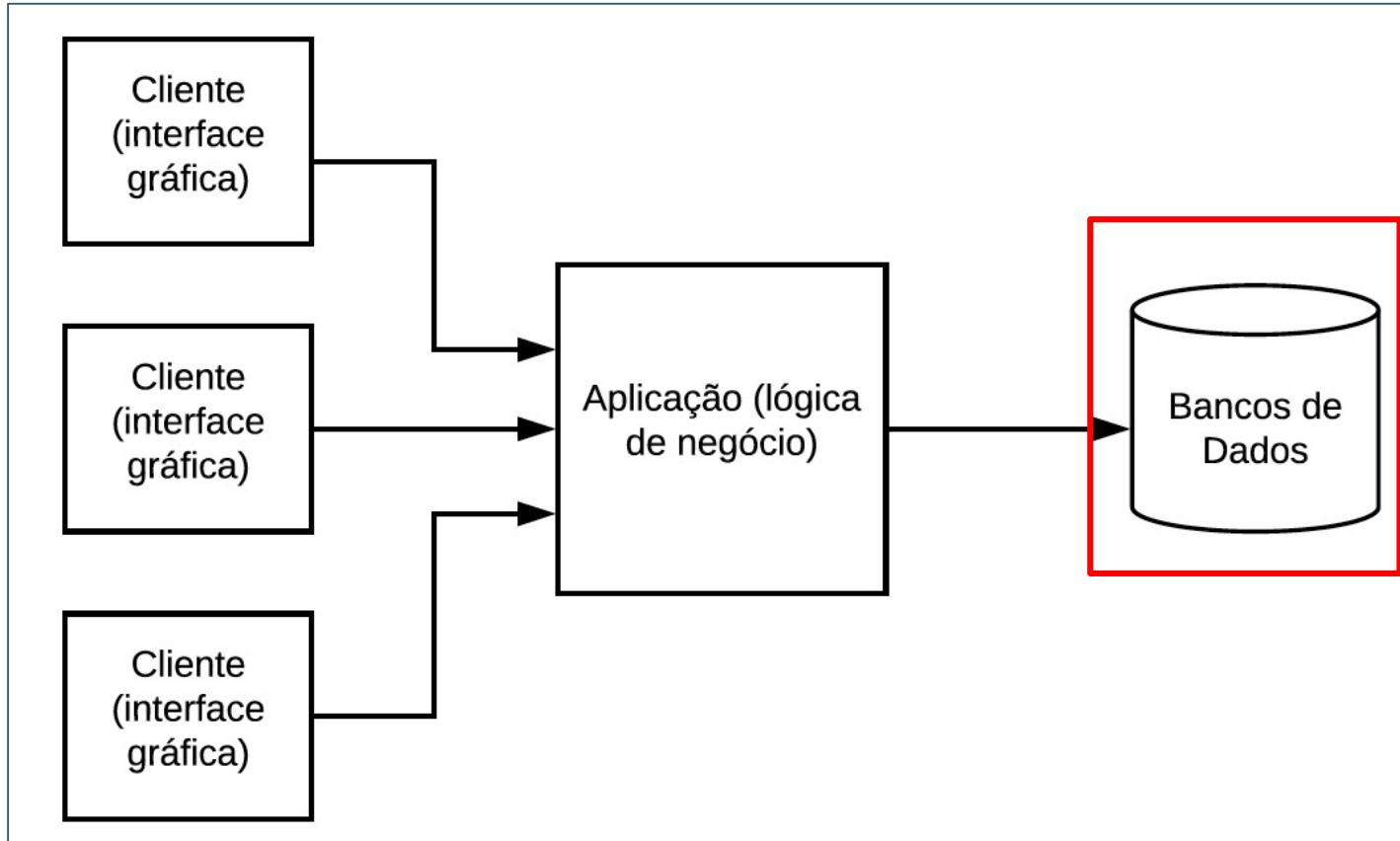
ARQUITETURA EM TRÊS CAMADAS



ARQUITETURA EM TRÊS CAMADAS



ARQUITETURA EM TRÊS CAMADAS



ARQUITETURA EM DUAS CAMADAS

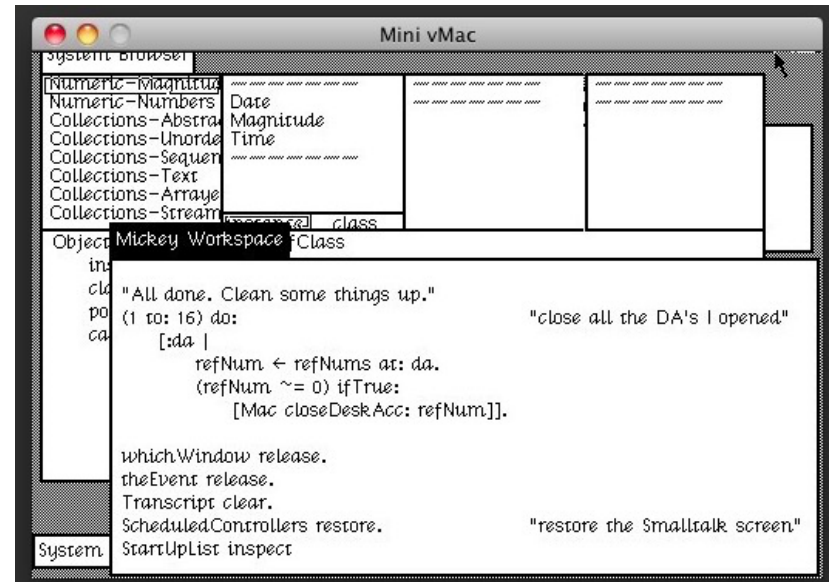
- Mais simples:
 - ← Camada 1 (cliente): interface + lógica
 - ← Camada 2 (servidor de BD): bancos de dados
- Desvantagem: todo o processamento é feito no cliente



ARQUITETURA MODEL-VIEW-CONTROLLER (MVC)

ARQUITETURA MVC

- Surgiu na década de 80, com a linguagem Smalltalk
- Proposta para implementar interfaces gráficas (GUIs)

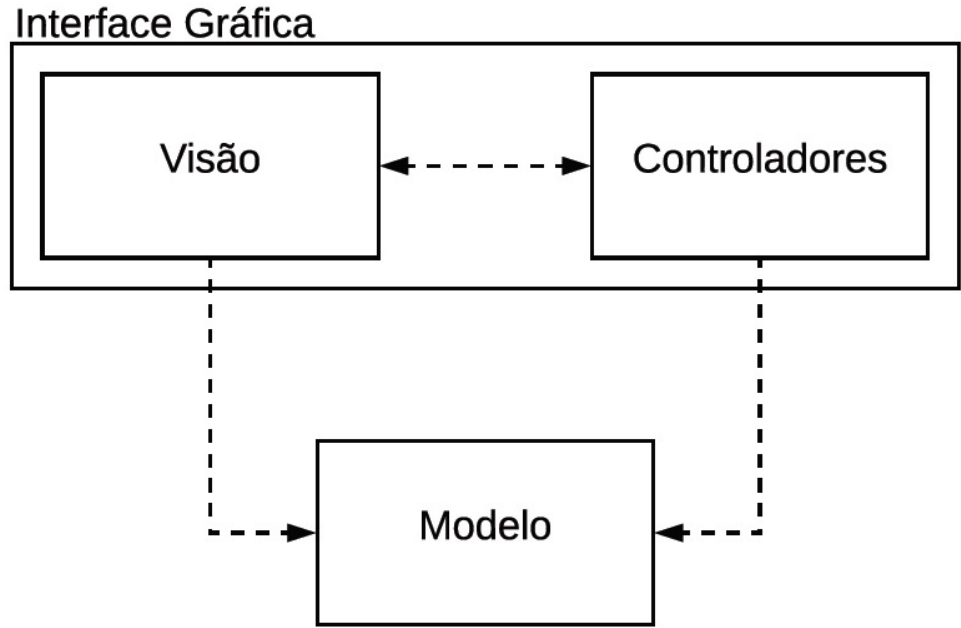


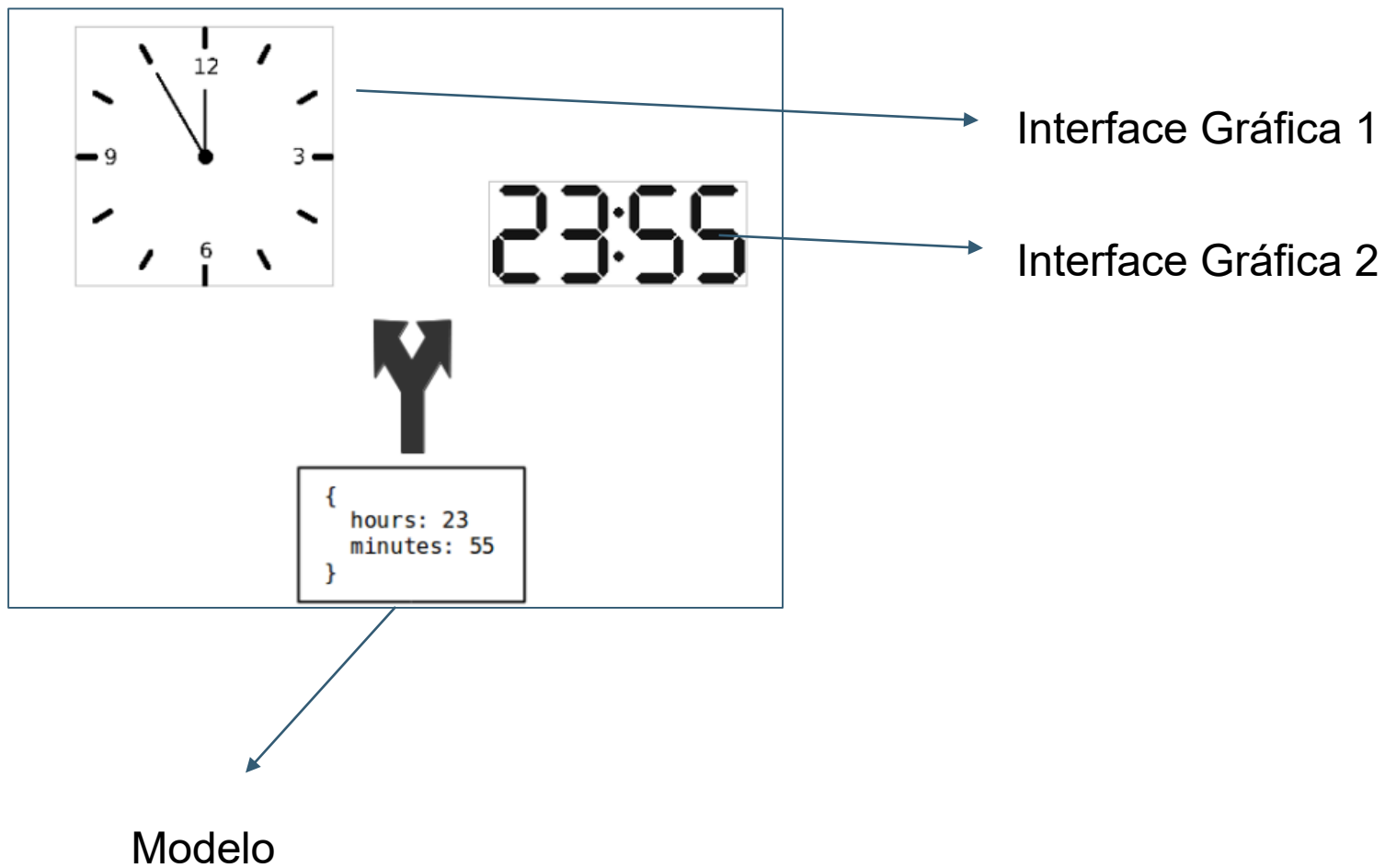
MVC

- Propõe dividir as classes de um sistema em 3 grupos:
 - ← **Visão:** classes para implementação de GUIs, como janelas, botões, menus, barras de rolagem, etc
 - ← **Controle:** classes que tratam eventos produzidos por dispositivos de entrada, como mouse e teclado
 - ← **Modelo:** classes com lógica e dados da aplicação

MVC = (VISÃO + CONTROLADORES) + MODELO

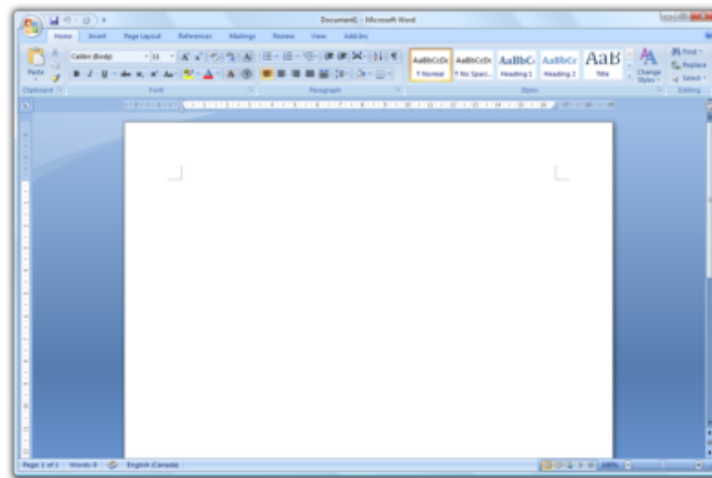
= INTERFACE GRÁFICA + MODELO





IMPORTANTE

- MVC não foi pensado para aplicações distribuídas; mas para aplicações desktop "monolíticas"
- Exemplo: Microsoft Word



MVC NOS DIAS DE HOJE

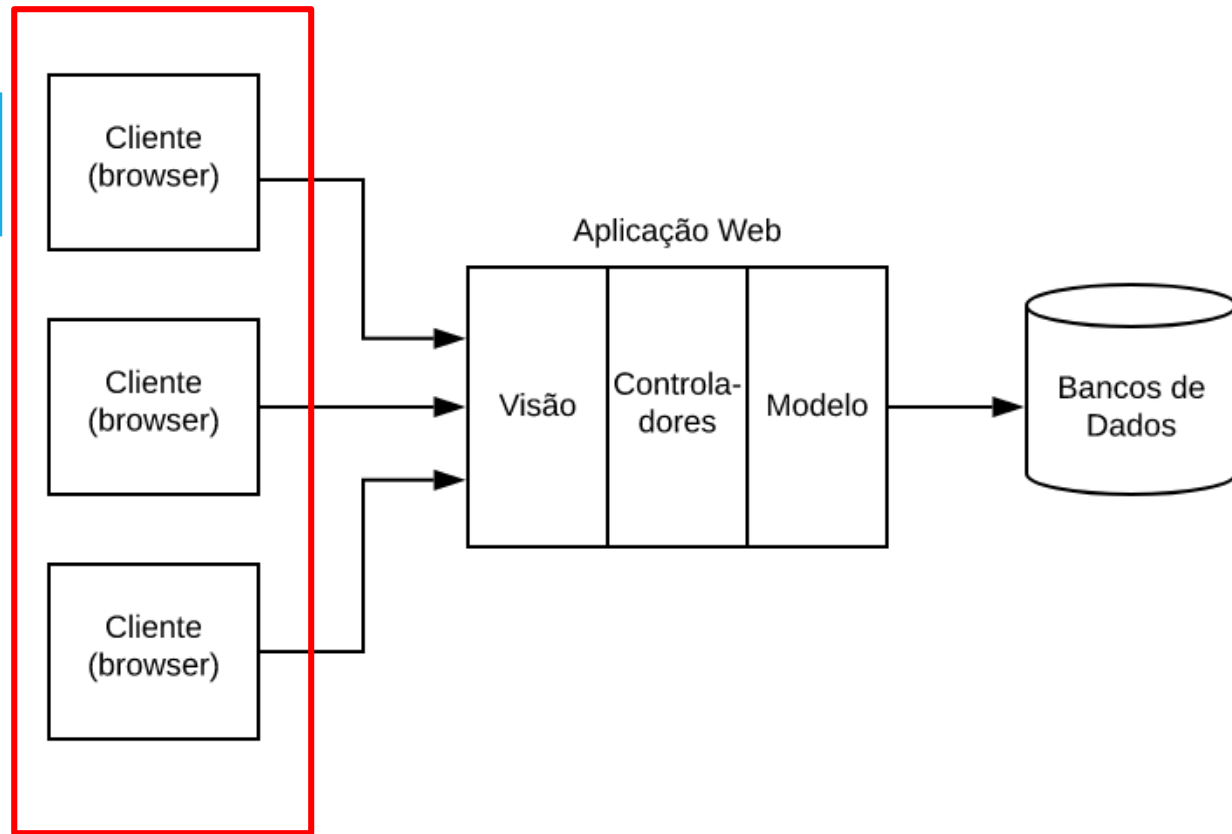
- MVC Web
- Single Page Applications

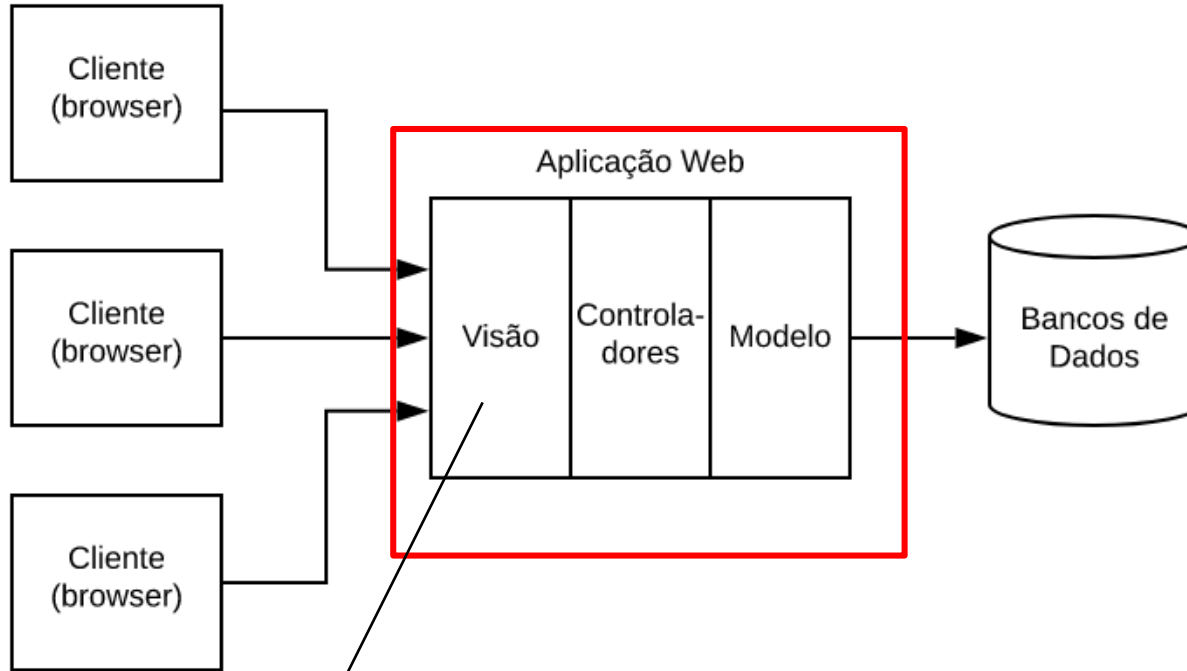


MVC WEB

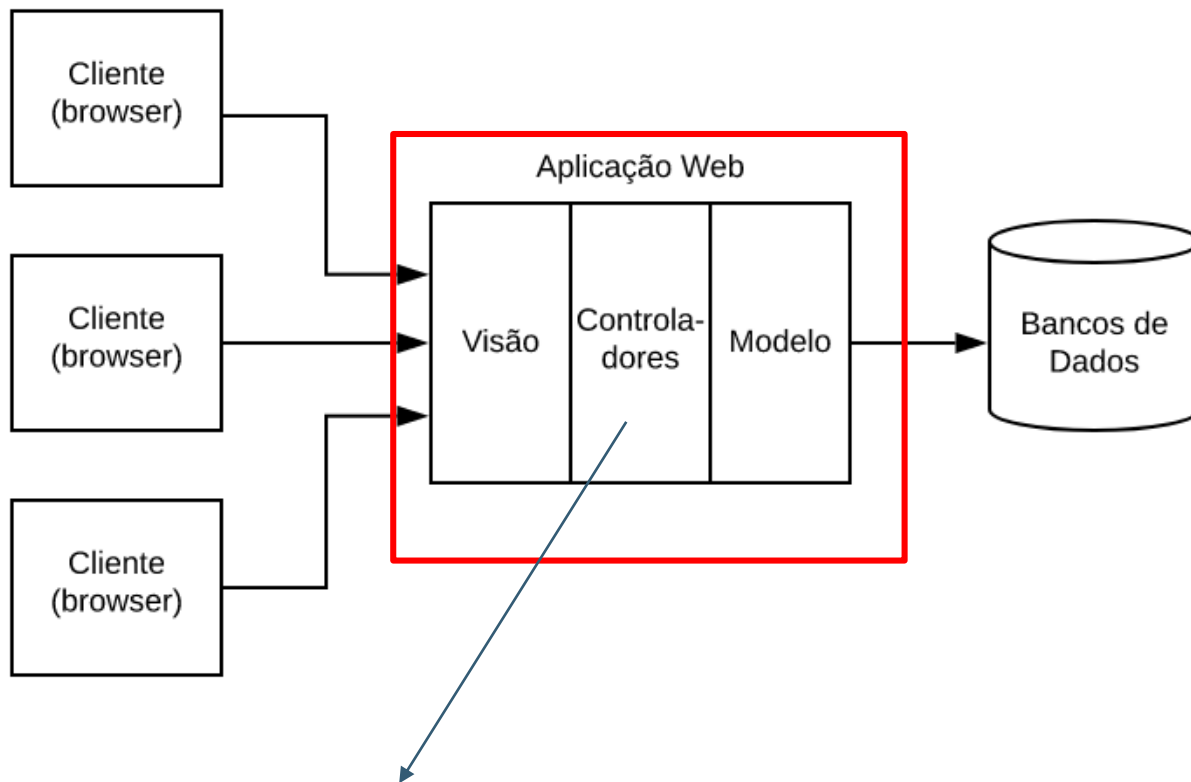
MVC WEB

- Adaptação de MVC para Web, ou seja, para sistemas distribuídos
- Usando frameworks com Ruby on Rails, Django, Spring, CakePHP, etc

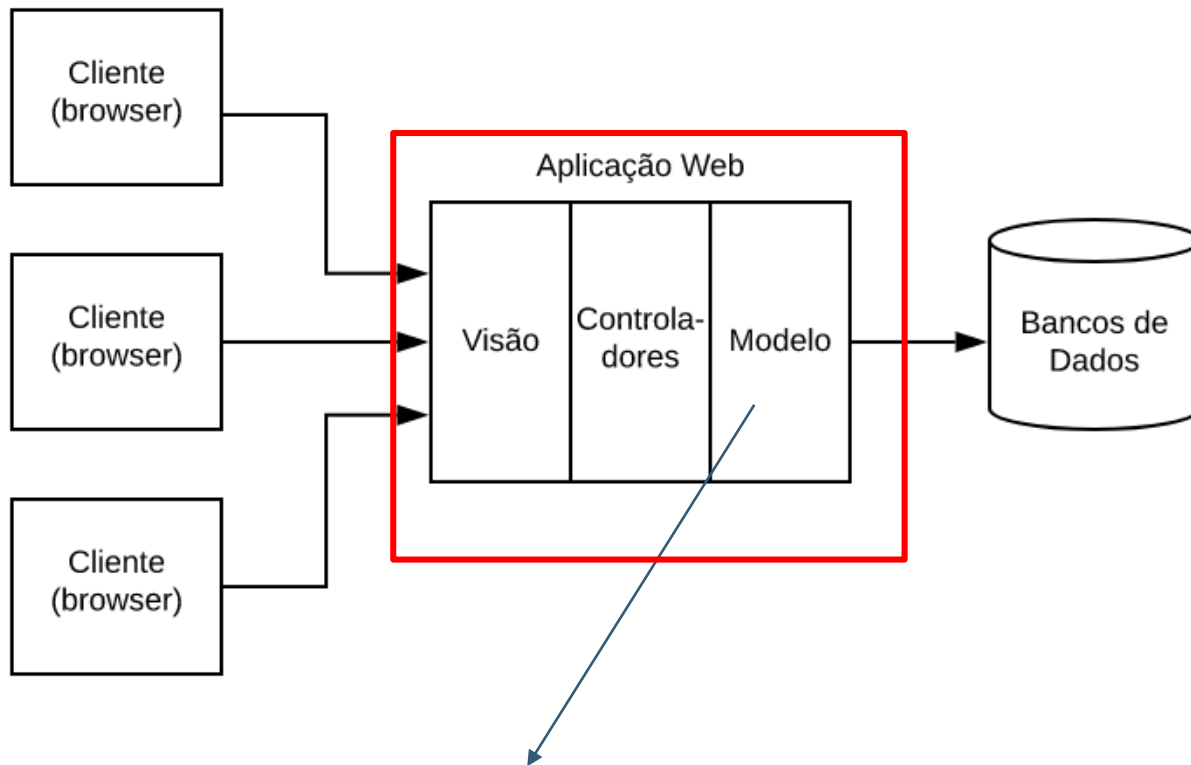




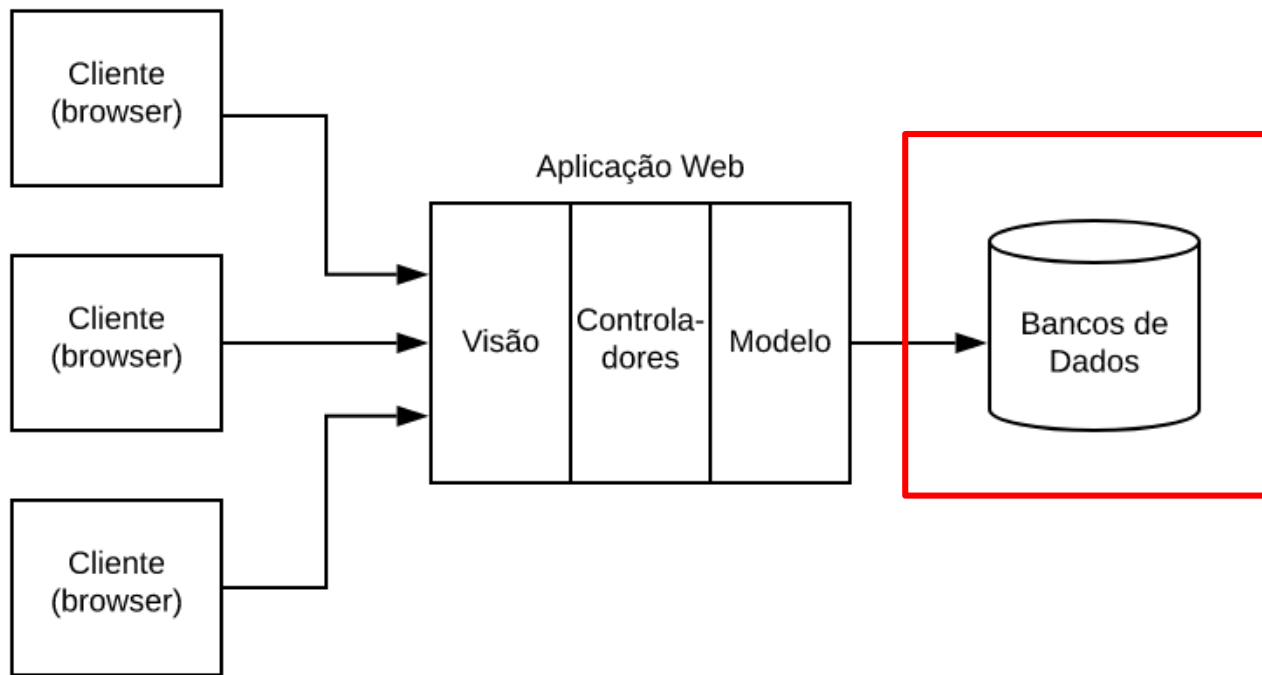
Páginas HTML, CSS, JavaScript
(o que o usuário vai ver)



Recebem dados de entrada e fornecem informações para páginas de saída

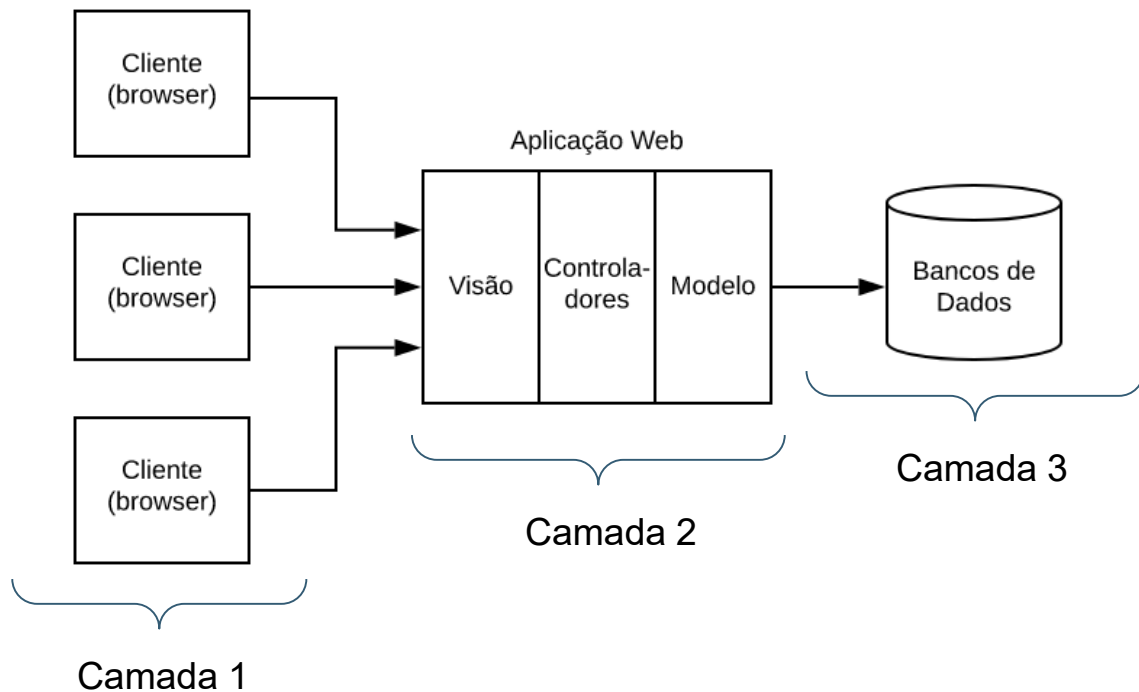


Lógica da aplicação (regras de negócio) e fazem a interface com o banco de dados



MVC WEB VS 3 CAMADAS

- O nome é MVC Web, mas parece com 3 camadas



EXEMPLO (MUITO SIMPLES, MAS DIDÁTICO) DE UM SISTEMA MVC WEB:

[HTTPS://REPLIT.COM/@ENGSOFTMODERNA/EXEMPLOARQUITETURAMVC](https://replit.com/@engsoftmoderna/exemploarquiteturamvc)



SINGLE PAGE APPLICATIONS (SPAS)

APLICAÇÕES WEB TRADICIONAIS

- Funcionam usando um modelo request-response:
 - ← Cliente requisita uma página ao servidor (request)
 - ← Servidor envia página e cliente a exibe (response)
 - ← Cliente solicita nova página (request)
 - ← etc
- Problema: interface menos responsivas, mais lentas, etc

SINGLE PAGE APPLICATIONS

- Aplicação que roda no browser, mas que é mais independente do servidor e "menos burra"
- "Menos burra": manipula sua própria interface e armazena os seus dados
- Mas pode acessar o servidor para buscar mais dados
- Exemplo: GMail, Google Docs, Facebook, Figma, etc
- Implementadas em JavaScript



EXEMPLO: APLICAÇÃO SIMPLES USANDO VUE.JS

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Interface (Web)
HTML

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa  
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Uma Simples SPA

Temperatura: 60

Incrementa

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Modelo

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

Modelo

Dados

Métodos


```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa
```

```
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

```
<h3>Uma Simples SPA</h3>
```

```
<div id="ui">
```

```
  Temperatura: {{ temperatura }}
```

```
  <p><button v-on:click="incTemperatura">Incrementa  
  </button></p>
```

```
</div>
```

```
<script>
```

```
var model = new Vue({
```

```
  el: '#ui',
```

```
  data: {
```

```
    temperatura: 60
```

```
  },
```

```
  methods: {
```

```
    incTemperatura: function() {
```

```
      this.temperatura++;
```

```
    }
```

```
  }
```

```
})
```

```
</script>
```

RESUMO

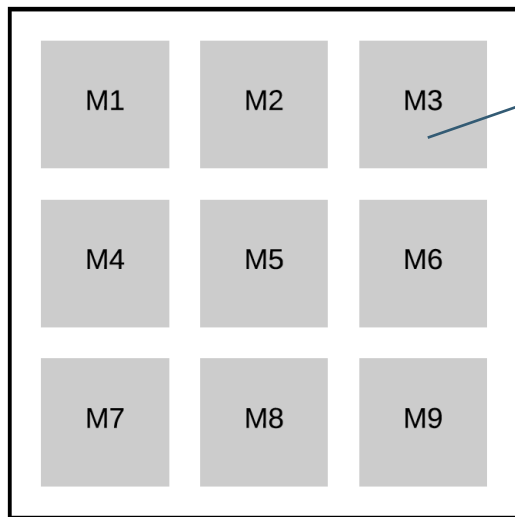
- MVC Tradicional (Smalltalk): aplicações gráficas, pré Web
- MVC Web: apesar do nome, lembra muito 3 camadas
- SPA: apesar de não ter no nome, lembra MVC tradicional



ARQUITETURAS BASEADAS EM MICROSERVIÇOS

VAMOS COMEÇAR COM MONOLITOS

- **Monolitos:** em tempo de execução, sistema é um único processo
(processo aqui é processo de sistema operacional)

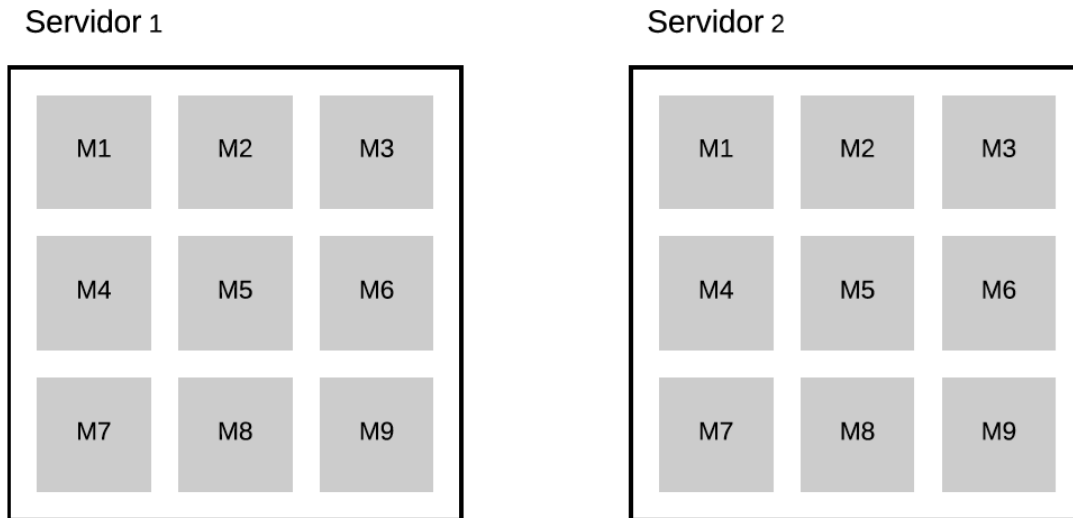


Módulos (de implementação e compilação)

Mas em tempo de execução, tudo roda em **único processo ou serviço**

PROBLEMA #1 COM MONOLITOS: ESCALABILIDADE

- Deve-se escalar o monolito inteiro, mesmo quando o gargalo de desempenho está em um único módulo



PROBLEMA #2 COM MONOLITOS: RELEASE É MAIS LENTA

- Processo de release é lento, centralizado e burocrático
- Times não tem poder para colocar módulos em produção
- Motivo: mudanças em um módulo podem impactar módulos que já estejam funcionando
- Acabam existindo:
 - ← Datas pré-definidas para release
 - ← Processo de "homologação"

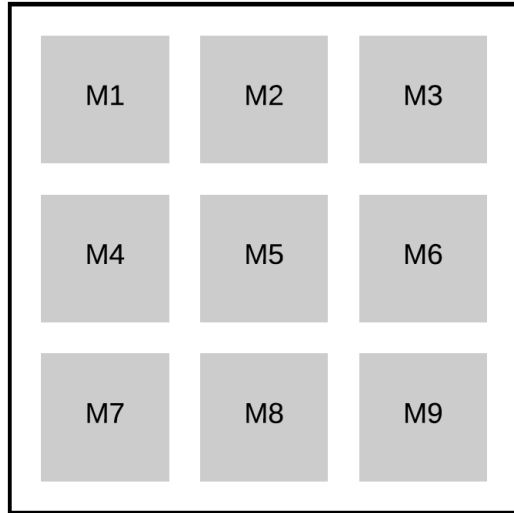


MICROSSERVIÇOS

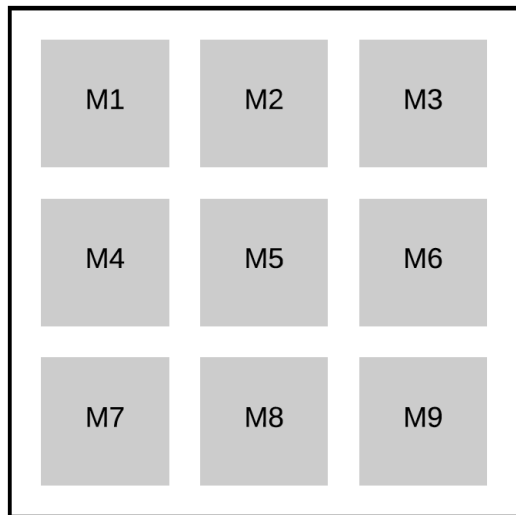
MICROSSERVIÇOS

- Módulos (ou conjuntos de módulos) viram processos independentes em tempo de execução
- Esses módulos são menores do que de um monolito
- Daí o nome **microsserviço**

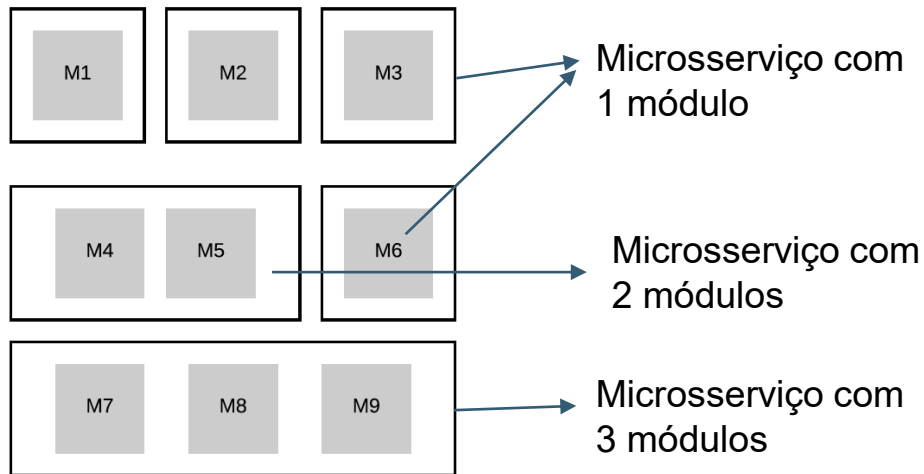
Arquitetura Monolítica



Arquitetura Monolítica



Arquitetura baseada em Microserviços

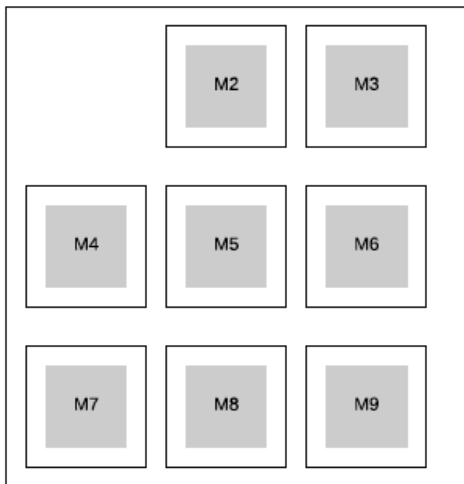


microserviço = processo (run-time, sistema operacional)

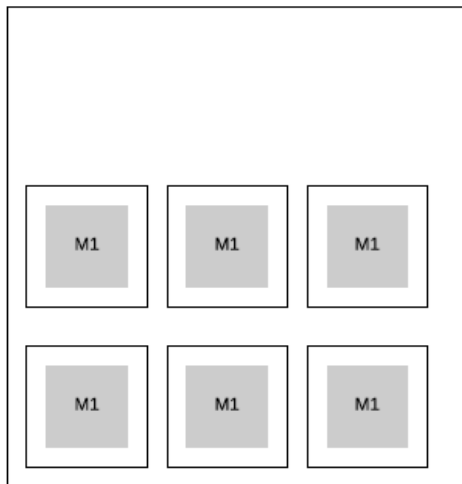
VANTAGEM #1: ESCALABILIDADE

- Pode-se escalar apenas o módulo com problema de desempenho

Servidor 1



Servidor 2



Só contém instâncias de M1, pois apenas ele é o gargalo de desempenho

VANTAGEM #2: FLEXIBILIDADE PARA RELEASES

- Times ganham autonomia para colocar microsserviços em produção
- Processo = espaço de endereçamento próprio
- Chances de interferências entre processos são menores

OUTRAS VANTAGENS

- Tecnologias diferentes
- Falhas parciais. Exemplo: apenas o sistema de recomendação pode ficar fora do ar

QUEM USA MICROSERVIÇOS?

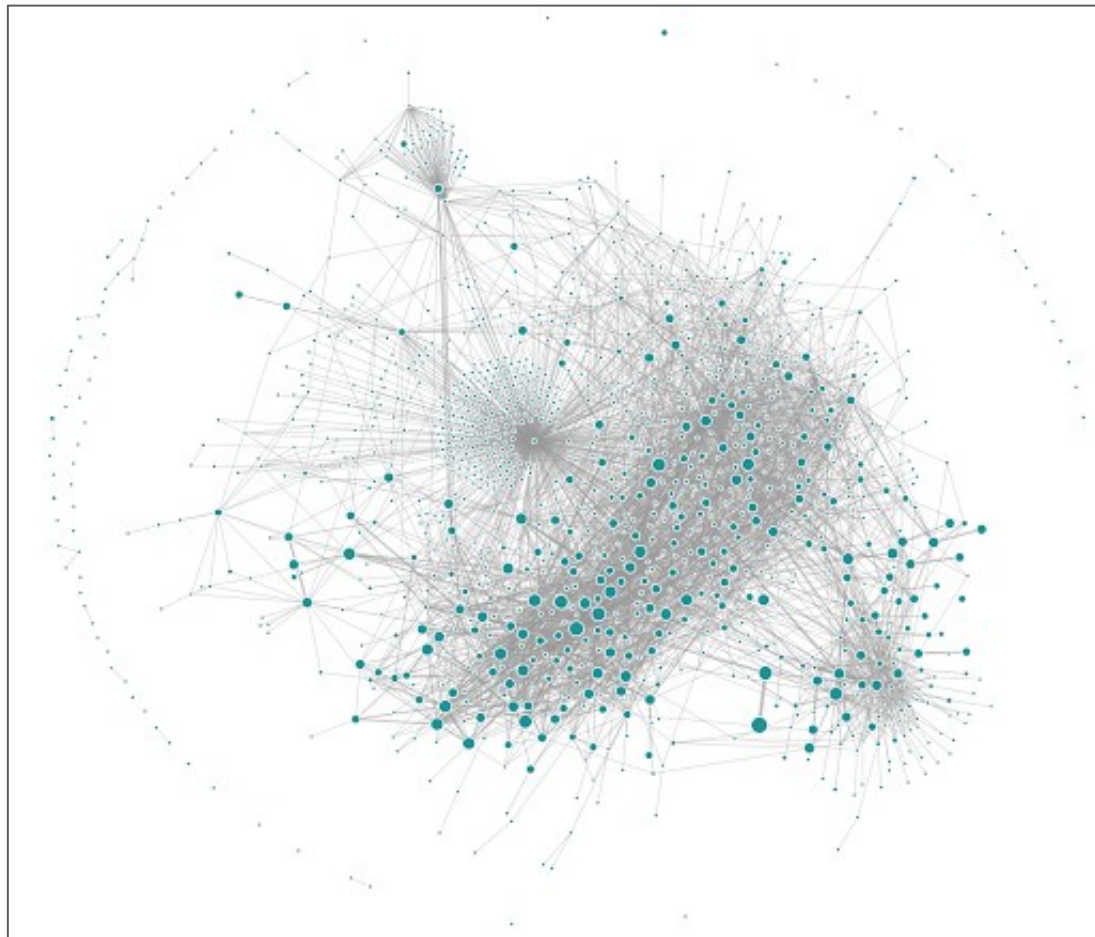
- Grandes empresas como Netflix, Amazon, Google, etc



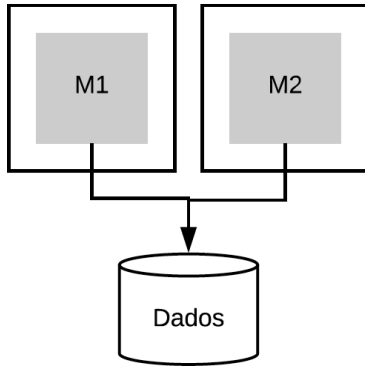
Cada nodo é um microserviço

EXEMPLO:

UBER (~2018)

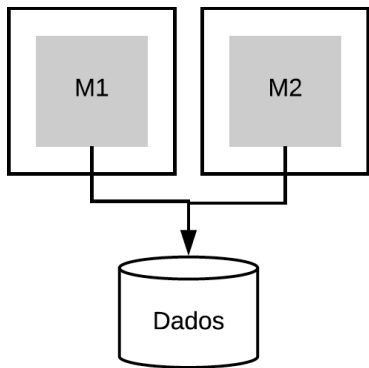


GERENCIAMENTO DE DADOS COM MICROSERVIÇOS

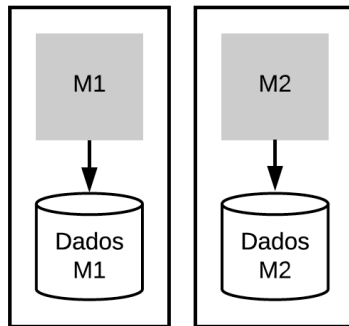


Arquitetura que **não** é recomendada.
Motivo: aumenta acoplamento entre M1
e M2

GERENCIAMENTO DE DADOS COM MICROSERVIÇOS



Arquitetura que não é recomendada.
Motivo: aumenta acoplamento entre M1 e M2



Arquitetura recomendada. Motivo: não existe acoplamento de dados entre M1 e M2. Logo, M1 e M2 podem evoluir de modo independente.
Se M1 precisar usar serviços de M2 (ou vice-versa), isso deve ocorrer via interfaces

DESVANTAGENS

- Arquitetura com microsserviços é mais complexa
 - ↩ Sistema distribuído (gerenciar centenas de processos)
 - ↩ Latência (comunicação é via rede)
 - ↩ Transações distribuídas

UMA RECOMENDAÇÃO



DHH  @dhh · Sep 28, 2019

This is why microservices can make sense for large companies with many, separate, and isolated teams. And why it just about never makes any sense for small companies where teams can grasp the entire application.



Programming Wisdom @CodeWisdom · Sep 28, 2019

"Organizations which design systems are constrained to produce designs which are copies of the communication structures of these organizations." -
Melvin Conway

 60

 570

 1.9K



OUTRA RECOMENDAÇÃO

- Começar com monolitos e pensar em microsserviços quando:
 - ↩ O monolito apresentar problemas de desempenho
 - ↩ O monolito estiver atrasando o processo de release
- Migração pode ser gradativa (microsserviços gradativamente extraídos do monolito)

REFERÊNCIAS

- PRESSMAN, R. S.. Engenharia de Software. Makron Books. 1995.
- Valente, M. T. Engenharia de Software Moderna. Editora Independente, 2020 (<https://engsoftmoderna.info/>)